

Lab Assignment 1 Report

Shariq Moghees

OL41895

Prof. Kidd

3-6-25

Program Structure

For this project, students were tasked to create a NASM assembly program that calculates the bitwise Hamming distance between two strings. Both strings may not be longer than 255 characters, and the longer string should be truncated so that both strings have an equal number of characters.

Data Section:

To kick things off, I set up all the data I'd need for the program. This included strings for the prompts (so I could ask the user to enter each string), plus a newline character. I also made space for the two input strings—`str1` and `str2`—a single character to store the final Hamming distance (`hamming_char`), and two bytes to hold the lengths of each string (`len1` and `len2`).

User Input:

Once that was ready, I printed the first prompt (`prompt1`) and read the first string into `str1`. Then I did the same for `prompt2` and `str2`. Both inputs were read using Linux system calls (`sys_read`), which let me grab up to 256 characters for each string (255 characters + 1 newline).

Calculating String Length:

With the strings in hand, I called `calc_len`, a simple subroutine that loops through each string until it hits a newline character. That gave me the length of each string, which I stored in `len1` and `len2`.

Length Comparison and Truncation:

The assignment required handling strings of different lengths, so if one string was longer than the other, I had to trim it down to match the shorter one. To do this, I compared `len1` and `len2`, and

whichever value was smaller got saved into `ecx`. That way, my Hamming distance loop only ran for the number of characters both strings actually had in common.

Hamming Distance Calculation:

This was the core of the whole program. I looped through the characters of both strings one-by-one and compared them using `xor`, checking for any differing bits. Each time a bit didn't match, I incremented a counter. This gave me the final Hamming distance.

Final Output:

Lastly, I converted that Hamming distance into an ASCII character by adding '0' to the count, and printed the result using `sys_write`.

Errors/Debugging:

While writing this program, I ran into only one issue. When the distance was greater than 9, the output always showed a random character instead of the correct HD. I tried several things to debug the issue. First, I reviewed how I was storing and printing the distance, thinking maybe I had an off-by-one error or a bad memory access. I also added some test cases with known Hamming distances (like comparing `foo` and `bar`) to check if the math was right.

Since the output looked like garbage data, I suspected it might be caused by accidentally reading or writing outside of the reserved space. I experimented with changing the size of `hamming_char` to `hamming_chars resb 4` to hold multi-digit numbers, which caused even more problems like segmentation faults. Ultimately, I realized I was only handling single-digit distances correctly, and anything more significant needed proper decimal conversion — something I didn't initially account for. If I had more time, I would rewrite that part to convert the number to a proper string before printing.

Code:

```
section .data
    newline db 10 ; newline character
    prompt1 db "Enter first string: ", 0
    prompt2 db "Enter second string: ", 0

section .bss
    str1 resb 256; reserve 255 bytes for each string +1 for newline
    str2 resb 256;
    hamming_char resb 1; reserve space for 1 character

    len1 resb 1 ; str length variables
    len2 resb 1

section .text
    global _start
_start:

    ;Prompt for first string
    mov eax, 4 ;sys_write
    mov ebx, 1 ;file descriptor stdout
    mov ecx, prompt1
    mov edx, 21 ;length of mesg
    int 0x80

    ;input processing for str1
    mov eax, 3 ;sys_read
    mov ebx, 0 ;stdin
    mov ecx, str1
    mov edx, 256 ;max characters +1 for newline
    int 0x80; kernel

    ;Calculate len1
    lea ecx, [str1]
    call calc_len
    mov [len1], al ;store length in len1

    ;Prompt for second string
    mov eax, 4 ;sys_write
```

```

mov ebx, 1 ;file descriptor stdout
mov ecx, prompt2
mov edx, 21 ;length of msg2
int 0x80

;input processing for str2
mov eax, 3 ;sys_read
mov ebx, 0 ;stdin
mov ecx, str2
mov edx, 256 ;max characters +1 for newline
int 0x80; kernel

;Calculate len2
lea ecx, [str2]
call calc_len
mov [len2], al ;store length in len2

;Compare len1 and len2 and truncate longer one
mov al, [len1]
mov bl, [len2]
cmp al, bl ;compare both
jbe use_len1 ;len 1 is shorter
movzx ecx, bl ;len 2 ia shorter
jmp len_set

use_len1:
    movzx ecx, al

len_set:
    ; Initialize pointers to strings
    mov esi, str1
    mov edi, str2

    ; Zero out hamming distance counter
    xor ebx, ebx

compare_loop:
    ; Load a character from each string
    mov al, [esi]
    mov dl, [edi]

```

```

; XOR to find differing bits
xor al, dl

; zero out edx
xor edx, edx

count_bits:
    shr al, 1 ; shift right, moving lowest bit into CF
    adc dl, 0 ; add CF to dl
    test al, al ; check if byte is empty (no 1s left)
    jnz count_bits ; loop until byte is empty

; Add result to overall Hamming Distance (in ebx)
add ebx, edx

; Move to next characters by incrementing char pointers
inc esi
inc edi
loop compare_loop

; Convert final Hamming Distance to ASCII ('0' + distance)
add bl, '0'

; Store ASCII character in reserved space
mov [hamming_char], bl

; print
mov eax, 4 ;sys_write
mov ebx, 1 ;file descriptor stdout
mov ecx, hamming_char
mov edx, 1 ;length (1 byte for one number)
int 0x80

; Exit program
mov eax, 1
xor ebx, ebx
int 0x80

```

```

calc_len:

```

```
    xor eax, eax ;zero out
len_loop:
    cmp byte [ecx+eax], 10
    je len_done
    inc eax
    cmp eax, 255
    je len_done
    jmp len_loop ;repeat for next character
len_done:
    ret
```

Output Test Cases:

```
[shariqml@linux5 Hamming_Distance]$ ./a.out
Enter first string: foo
Enter second string: bar
8
[shariqml@linux5 Hamming_Distance]$ ./a.out
Enter first string: abc
Enter second string: abcd
0
[shariqml@linux5 Hamming_Distance]$ ./a.out
Enter first string: Hello
Enter second string: Hi
2
[shariqml@linux5 Hamming_Distance]$
```